Projektbeschreibung Wahlfach Autonomes Fahren

Jonas Weihing

19.10.2026

1 Projektbeschreibung

Dieses Projekt setzt eine modulare Pipeline für ein autonom fahrendes Auto in der *Gymnasium Car Racing Simulation*[1] um. Die Pipeline besteht aus den folgenden Modulen und baut auf den in der Vorlesung vermittelten Inhalten auf:

1. Fahrspurerkennung
2. Pfadplanung
3. Querregelung
4. Längsregelung

Für die Umsetzung muss das beigefügte Framework verwendet werden, das bereits mehrere Testdateien und eine Grundstruktur zur Verfügung stellt. Die `README.md` erläutert dabei, wie alle notwendigen Bibliotheken installiert werden können und wo sich welche Datei befindet.

2 Module

Im Folgenden werden die einzelnen Pipelinemodule erläutert. Die beigefügten Tipps dienen als Orientierung, müssen jedoch nicht umgesetzt werden. Zusätzlich kann jedes Modul unabhängig über einzelne Test-Skripte getestet werden. Je nach Implementierung kann es notwendig sein, die *run*-Methode des jeweiligen Test-Skripts anzupassen. Am Ende jedes Moduls finden Sie Vorschläge für optionale Erweiterungen.

2.1 Fahrspurerkennung

Im ersten Modul der Pipeline wird die Fahrspurerkennung durchgeführt. Ziel ist es, die linke und rechte Begrenzung der Fahrspur zu erkennen und zuzuordnen. Je robuster und zuverlässiger die Erkennung ist, desto besser können die nachfolgenden Module darauf aufbauen.

Ziel: Erkennen und Zuordnen der linken und rechten Fahrspurbegrenzungen.

Tipps:

- Erkannte Fahrspuren können als Pixelkoordinaten oder mathematische Funktion dargestellt werden (z. B. Polynom oder Spline).
- Empfohlenes Vorgehen:
  1. **Bildvorverarbeitung:** Zuschneiden des Bildes und Maskierung des Fahrzeugs, um Störungen zu minimieren.
  2. **Kantenerkennung:** Berechnen der Bildgradienten zur Hervorhebung der Übergänge zwischen Fahrspur und Grünfläche.
  3. **Zuordnung:** Zuordnung der erkannten Kanten zur linken und rechten Fahrspurbegrenzung.
  4. **Funktionsanpassung:** Optional kann eine Funktion auf die zugeordneten Kanten angepasst werden, um eine mathematische Repräsentation zu erhalten.
- Testen Sie die einzelnen Komponenten in mehreren Situationen, um Fehler in der Perzeption frühzeitig zu finden.

[1] https://gymnasium.farama.org/environments/box2d/car_racing/

Testing: Um die Fahrspurerkennung zu testen, steht Ihnen das Skript *test_lane_detection.py* zur Verfügung. Das Auto kann mithilfe der Tastatur (WASD oder Pfeiltasten) in der Simulation gesteuert werden.

Optional:
- Spur auch für zufällige Farbwerte erkennen. Setzen Sie dafür den Parameter `domain_randomize` beim Ausführen der Testdatei auf `True`:
  `python main.py --domain_randomize`
- Verbesserungen, die über den in der Vorlesung besprochenen Ansatz hinausgehen und die Stabilität und Robustheit erhöhen (Bitte ausführlich dokumentieren und vorstellen).

2.2 Pfadplanung

Im zweiten Modul wird mithilfe der im ersten Modul erkannten Fahrspurbegrenzungen der zu fahrende Pfad geplant. Dies kann im einfachsten Fall die Mitte der Fahrspur sein, für höhere Geschwindigkeiten und optimierte Kurvenfahrten kann diese jedoch in Abhängigkeit von der Krümmung der Kurve angepasst werden.

Ziel: Berechnung eines gültigen Pfads bzw. einer gültigen Trajektorie.

Tipps:
- Verwenden Sie zunächst die Mitte der Fahrspur, das reicht für die Mindestanforderungen.
- Fahrspuren können entweder als Pfad (Menge an Punkten) oder Trajektorie (Funktion) beschrieben werden.
- Überlegen Sie sich, welches Koordinatensystem verwendet werden soll.
- Das Format der Fahrspur hat einen direkten Einfluss auf die Regelung.
- Die Krümmung der Kurve kann anhand des Kurvenradius oder der zweiten Ableitung berechnet werden.

Testing: Um die Pfadplanung zu testen, steht Ihnen das Skript *test_path_planning.py* zur Verfügung. Die Fahrspurbegrenzungen werden Ihnen aus der Simulation in Form von Pixelkoordinaten zur Verfügung gestellt, sodass Sie die Pfadplanung unabhängig von der Fahrspurerkennung entwickeln und testen können. Das Auto kann mithilfe der Tastatur (WASD oder Pfeiltasten) in der Simulation gesteuert werden.

Optional:
- Optimierung der Trajektorie anhand der Kurvenkrümmung.
- Weitere Verbesserungen, die die Performance maßgeblich beeinflussen (Bitte ausführlich dokumentieren und vorstellen).

2.3 Querregelung

Das dritte Modul der Pipeline widmet sich der Querregelung, also der Bestimmung des Lenkwinkels. Hierfür kann der bewährte Stanley-Regler verwendet werden, aber auch die Implementierung anderer Regler wie des Pure-Pursuit-Reglers ist möglich.

Ziel: Funktionierender Stanley- oder Pure-Pursuit-Regler, der das Fahrzeug zuverlässig in der Spur hält.

Tipps:
- Achten Sie auf das Format aus der Pfadplanung.
- Implementieren Sie zunächst den Stanley-Regler mit den Standardwerten.
- Die Parameter des Reglers lassen sich iterativ anpassen (am besten das Test-Skript verwenden).

Testing: Um die Querregelung zu testen, steht Ihnen das Skript *test_lateral_control.py* zur Verfügung. Der zu fahrende Pfad wird Ihnen aus der Simulation in Form von diskreten Wegpunkten zur Verfügung gestellt, sodass Sie die Querregelung unabhängig von der Fahrspurerkennung und Pfadplanung entwickeln und testen können. Die Geschwindigkeit des Autos kann mithilfe der Tastatur (W, S oder Pfeiltasten) gesteuert werden.

Optional:
- Implementieren und Vergleichen beider Regelungsansätze. Wichtig: Beide müssen lauffähig und stabil sein, sowie über einen Parameter aktiviert werden können.
- Entwicklung einer Dämpfung, die dem D-Anteil des PID-Reglers ähnelt.
- Weitere Verbesserungen, die die Performance maßgeblich beeinflussen (Bitte ausführlich dokumentieren und vorstellen).

2.4 Längsregelung

Das letzte Modul der Pipeline implementiert die Längsregelung, die für die Steuerung der Fahrzeuggeschwindigkeit zuständig ist. Dazu wird die aktuelle Geschwindigkeit von der Simulation bereitgestellt. Die Längsregelung besteht aus zwei Teilen. Zum einen muss die Zielgeschwindigkeit bestimmt werden, die sich idealerweise nach der Krümmung der Straße richtet. Zum anderen muss mithilfe des PID-Reglers die Geschwindigkeit des Fahrzeugs durch gezieltes Beschleunigen oder Bremsen geregelt werden.

Ziel: Dynamische PID-Längsregelung, welche die Geschwindigkeit des Fahrzeugs an die Fahrsituation anpasst.

Tipps:
- Bremse und Gas müssen an die Simulation separat übertragen werden.
- Fangen Sie langsam an, es handelt sich um ein Rennauto.
- Bremsen bzw. Gasgeben und gleichzeitig starkes Lenken sollten Sie vermeiden, Sie werden sehen, warum.
- Ermitteln Sie die Zielgeschwindigkeit anhand der Kurvenkrümmung.

Testing: Um die Längsregelung zu testen, steht Ihnen das Skript *test_longitudinal_control.py* zur Verfügung. Die Lenkung des Fahrzeugs muss dabei händisch gesteuert werden, sodass die Längsregelung unabhängig von der Fahrspurerkennung, Pfadplanung und Querregelung entwickelt und getestet werden kann. Die Lenkung des Autos kann mithilfe der Tastatur (A, D oder Pfeiltasten) gesteuert werden.

Optional:
- Verbesserte Berechnung der Zielgeschwindigkeit, z. B. durch weitere Heuristiken.
- Weitere Verbesserungen, die die Performance maßgeblich beeinflussen (Bitte ausführlich dokumentieren und vorstellen).

3 Bewertung und Abgabe

3.1 Abgabe

Die Abgabe erfolgt gruppenweise über **Moodle** bis zum **23.11.2026 um 23:59 Uhr** als **ZIP-Datei**. Diese muss enthalten:

- den vollständigen, lauffähigen Python-Code der Pipeline,
- eine `README.md`, die beschreibt, was umgesetzt wurde und, falls zusätzliche Features oder Reglervarianten implementiert wurden, wie diese verwendet werden (z. B. Parameter und Aufrufe).

Was nicht dokumentiert ist, kann nicht bewertet werden. Dies gilt insbesondere für optionale Erweiterungen.

3.2 Präsentation und Fragen

Zusätzlich zur Abgabe präsentieren Sie Ihr Projekt in einer **10-minütigen Präsentation**, an die sich **10 Minuten Fragen** anschließen. Die Präsentationen finden in Präsenz am **30.11.2026** statt, ein Ausweichtermin bei Verhinderung ist der **01.12.2026**. Der Raum wird noch bekannt gegeben. Für die Präsentation gibt es Zeitslots, in die Sie sich eintragen müssen. In der Präsentation sollen Sie darstellen:

- **was** Sie umgesetzt haben,
- **wie** Sie dabei vorgegangen sind,
- **warum** Sie sich für diese Umsetzung entschieden haben (inklusive betrachteter Alternativen),
- welche **Ergebnisse** und Vergleiche sich ergeben haben, z. B. Scores, Vergleich von Reglern oder Parametern, sowie Grenzen Ihrer Lösung.

In der anschließenden Fragenrunde wird geprüft, ob Sie Ihre Lösung verstanden haben und die Umsetzung selbst erarbeitet haben. Die Fragen beziehen sich überwiegend auf Ihr Projekt und können vereinzelt auch über das Projekt hinausgehen und Inhalte der Vorlesung betreffen. Es handelt sich ausdrücklich nicht um eine mündliche Prüfung, in der theoretisches Wissen abgefragt wird.
Der Einsatz von KI-Werkzeugen ist erlaubt. Sie müssen den entstandenen Code jedoch vollständig erklären und Ihre Entscheidungen begründen können.

3.3 Bewertung

Die Gesamtbewertung setzt sich zu je **50 %** aus dem Projekt und der Präsentation mit Fragenrunde zusammen. Die Details der einzelnen Kriterien finden Sie im bereitgestellten Bewertungsbogen. Die folgende Übersicht zeigt die Kriterien mit ihrem Anteil an der Gesamtbewertung in Prozent:

- **Projekt (50 %):** Funktionalität der Pipeline (15), Umsetzung der Module (15), Dokumentation (10), Codequalität und Laufzeit (5), Originalität und Kreativität (5).
- **Präsentation (50 %):** Fragenrunde: Verständnis und Eigenleistung (15), Darstellung der Umsetzung (10), Begründung der Entscheidungen (10), Ergebnisse, Vergleiche und Reflexion (10), Vortragsqualität und Zeiteinhaltung (5).

Mindestanforderung. Für die Basisanforderung sollte das Auto mit einer moderaten Geschwindigkeit auf der Strecke fahren und dabei einen Score von mindestens 300 erreichen. Der Score wird als Mittelwert über **50 Seeds** gemessen. Zur Orientierung dienen Referenzimplementierungen der Lehrveranstaltung: Eine Minimallösung erreicht einen Score von etwa 400, eine vollständig KI-generierte Lösung einen Score von über 900 und eine stark optimierte Lösung einen Score von bis zu 950.

Erlaubte Bibliotheken. In der Pipeline sind ausschließlich die Bibliotheken `numpy` und `scipy` erlaubt. `OpenCV` dürfen Sie nur für die Darstellung und das Debugging einsetzen. Die Bildverarbeitung sollen Sie selbst mit einfachen `numpy`-Methoden implementieren. Setzen Sie `OpenCV` in der Pipeline für die Bildverarbeitung oder andere Aufgaben ein, die **nicht** zum Debugging gehören, ist für *Umsetzung der Module* höchstens die Bewertung *ausreichend* möglich.

Erweiterungen. Erweiterungen sind optional. Eigene Ideen und Ansätze, die über die Standardlösungen hinausgehen, fließen in das Kriterium *Originalität und Kreativität* ein. Dieses Kriterium betrachtet die gesamte Pipeline, nicht nur gezielte Erweiterungen. Beispiele sind die Erweiterung der Regler über die Standardregler hinaus oder die Entwicklung einer besonders robusten Fahrspurerkennung. Tipps für optionale Verbesserungen sind in den Modulbeschreibungen genannt. Funktionierende Zusatzimplementierungen werden zusätzlich im Kriterium *Umsetzung der Module* berücksichtigt und können dort zur Bestnote beitragen. Eigene Ideen und Erweiterungen müssen ausführlich dokumentiert sein und in der Präsentation vorgestellt werden.

Individuelle Bewertung. Falls eine individuelle Bewertung gewünscht wird, muss dies im Vorfeld mitgeteilt werden. Ohne entsprechende Ankündigung wird eine Gruppenbewertung vorgenommen. Für eine individuelle Bewertung muss im Code klar gekennzeichnet sein, was von wem entwickelt wurde.